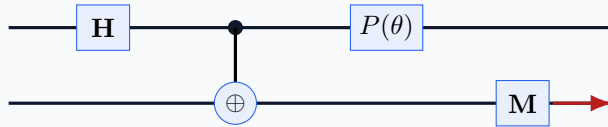


QPU Static Circuit Compiler

Complete Protocol, Gate, and Truth-Table Reference



Bailey Forbes
September 13, 2026

This guide describes the current QPU language, gate set, compiler, simulator, and truth-table system. It distinguishes syntax that produces executable circuit gates from compatibility syntax that is only recorded in the compiler log.

Contents

1	Scope and execution model	5
1.1	Three levels of support	5
1.2	QPU instruction inventory	5
2	First circuit: from source to result	5
2.1	Read the program line by line	6
2.2	What one run means	6
2.3	Adding user-controlled input	6
3	Lexical and file syntax	6
3.1	Logical lines, whitespace, and case	6
3.2	Hash comments	7
3.3	Block comments	7
3.4	PARAMS:: the process input declaration	7
3.5	Protocol file forms	7
4	Names and token references	8
4.1	Token names	8
4.2	Direct references	8
4.3	Parameter references with a dollar sign	8
4.4	Cycle-qualified references with :	9
4.5	Aliases created by SET	9
4.6	Numeric workspace references	9
4.7	Process scope and child-local references	9
5	Built-in state constants	10
5.1	0p: the zero basis state	10
5.2	1p: the one basis state	10
5.3	sp: equal superposition	10
5.4	Dimension-suffixed constants	10
5.5	Constants and state parameters	11
6	Instruction flags and argument boundaries	11
6.1	-I: input and control references	11
6.2	-O: output and target references	11
6.3	Where a flag list ends	11
6.4	No-parameter-substitution marker	12
6.5	Positional arguments	12
7	Gate instruction grammar and arity	12
7.1	General gate shape	12
7.2	Why arity matters	12
7.3	Validation and common mistakes	13

8	Mathematical foundations	13
8.1	Real numbers and complex numbers	13
8.2	The complex plane, magnitude, and conjugate	13
8.3	Angles, radians, sine, and cosine	14
8.4	Euler’s formula and Euler’s identity	14
8.5	Vectors and basis vectors	14
8.6	Normalization and the Born rule	14
8.7	Matrices as gates	15
8.8	Unitarity and reversibility	15
9	Qubits, phase, and the Bloch sphere	15
9.1	A qubit is not just an unknown bit	15
9.2	Global phase and relative phase	15
9.3	Bloch-sphere coordinates	16
9.4	How gates move a Bloch vector	16
10	Multiple qubits and entanglement	16
10.1	Tensor products	16
10.2	Entanglement	17
10.3	Controlled operations	17
11	Measurement, probability, and truth tables	17
11.1	Projective measurement	17
11.2	Single runs and repeated runs	17
11.3	Truth tables versus quantum amplitudes	18
11.4	A complete interference example	18
12	Single-qubit primitive gates	18
12.1	Pauli X (X)	18
12.2	Pauli Y (Y)	19
12.3	Pauli Z (Z)	19
12.4	Hadamard (H)	19
12.5	S phase (S)	20
12.6	T phase (T)	20
12.7	Arbitrary phase ($PHASE$)	20
12.8	Backward-prefix syntax	21
13	Controlled and two-qubit primitive gates	21
13.1	Controlled NOT ($CNOT$)	21
13.2	Doubly controlled NOT / Toffoli ($CCNOT$)	22
13.3	Controlled Z (CZ)	22
13.4	Controlled Y (CY)	23
13.5	Swap ($SWAP$)	23
14	Derived logical gates	23

14.1 Logical NOT (NOT)	23
14.2 Reversible AND (AND)	24
14.3 Reversible NAND (NAND)	24
14.4 Reversible OR (OR)	25
14.5 Reversible XOR (XOR)	25
15 Measurement gate	26
15.1 Measurement (MEASURE)	26
16 State, token, and cycle commands	26
16.1 SET — initialize or alias (lowered/executed)	26
16.2 CREATETOKEN — reserve wires (lowered/executed)	26
16.3 DELETETOKEN — token lifetime marker (accepted, not executed)	26
16.4 FREE — releasable-memory marker (accepted, not executed)	27
16.5 INCREASECYCLE — cycle boundary (lowered/executed)	27
16.6 JOIN — composite-register marker (accepted, not executed)	27
16.7 SPLIT — component-register marker (accepted, not executed)	27
17 Process composition and return values	27
17.1 MAIN-PROCESS (lowered/executed)	27
17.2 DECLARECHILD (accepted, not executed)	28
17.3 RUNCHILD — expand a child process (lowered/executed)	28
17.4 CALL — alternate child invocation (lowered/executed)	28
17.5 RETURNVALS (lowered/executed)	28
17.6 ACCEPTVALS (lowered/executed)	28
17.7 Nested full-adder example	28
18 Accepted compatibility commands	29
18.1 COMPILEPROCESS (accepted, not executed)	29
18.2 MASTERVAL (accepted, not executed)	29
18.3 Save-state checkpoint instruction (accepted, not executed)	30
18.4 Load-state checkpoint instruction (accepted, not executed)	30
19 Truth-table companion syntax	30
19.1 Text .qpuio structure	30
19.2 MAIN-PROCESS: table header	30
19.3 INPUTS: and OUTPUTS: column groups	30
19.4 Column header row	31
19.5 Indexed data rows	31
19.6 CSV spelling	31
19.7 Partial truth tables	31
19.8 JSON .qpuio envelope	32
19.9 How a table test runs	32
19.10 Protected bundled tables	32

20 Complete worked protocol	32
21 Compiler output and lowering details	33
22 Error conditions and practical rules	33
23 Quick-reference grammar	34
24 Glossary	34
25 Operation checklist	35

1 Scope and execution model

The compiler turns a plain-text `.qpu cir` protocol into one flat sequence of circuit gates:

protocol source $\longrightarrow$ logical lines $\longrightarrow$ QPU instructions $\longrightarrow$ scoped tokens $\longrightarrow$ gates $\longrightarrow$ state-vector simulation.

Child processes are expanded at compile time. The UI and simulator therefore see the same ordered gate list even when the source contains nested processes.

1.1 Three levels of support

Status	Meaning
lowered/executed	The QPU language accepts the instruction and the compiler emits behavior that the simulator executes.
accepted, not executed	The QPU language accepts the instruction and preserves it in compiler output or the compiler log, but no visual/simulator operation is emitted yet.
compiler-internal	The compiler may emit the operation itself, but protocol source cannot name it directly. RESET is the important example.

1.2 QPU instruction inventory

Command	Status	Purpose
X, Y, Z, H, S, T, PHASE	lowered/executed	Single-qubit matrix operations.
CNOT, CCNOT, CZ, CY, SWAP	lowered/executed	Controlled and two-wire primitive operations.
NOT, AND, NAND, OR, XOR	lowered/executed	Derived logical target mutations.
MEASURE	lowered/executed	Measure one named wire or all known wires.
SET, CREATETOKEN	lowered/executed	Initialize/alias tokens and reserve wires.
RUNCHILD, CALL	lowered/executed	Expand a catalog/library child process.
RETURNVALS, ACCEPTVALS	lowered/executed	Bind child and process outputs.
MAIN-PROCESS, INCREASECYCLE	lowered/executed	Mark a process and flush cycle workspace.
FREE, DELETETOKEN, DECLARECHILD	accepted, not executed	Lifetime/declaration hints; currently logged.
JOIN, SPLIT	accepted, not executed	Register-memory compatibility syntax.
COMPILEPROCESS, MASTERVAL	accepted, not executed	Process-control compatibility syntax.
SAVE_STATE, LOAD_STATE	accepted, not executed	Checkpoint compatibility syntax.

Not source syntax: RESET exists in the gate registry, but it is not a supported source instruction. Write `SET wire 0p`; the compiler schedules internal RESET gates for zeroed workspace.

2 First circuit: from source to result

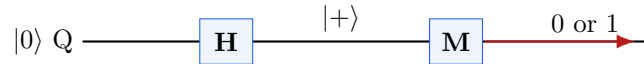
The shortest useful learning path is to prepare one qubit, transform it, measure it, and declare it as an output:

```
MAIN-PROCESS FirstCircuit
CREATETOKEN -I Q
SET Q 0p
H -I Q -O Q
```

```
MEASURE -I Q
RETURNVALS Q
```

2.1 Read the program line by line

1. MAIN-PROCESS FirstCircuit gives the protocol a name.
2. CREATETOKEN -I Q reserves a logical wire named Q.
3. SET Q 0p prepares Q in the basis state $|0\rangle$.
4. H -I Q -0 Q applies Hadamard to that same wire, producing $|+\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$.
5. MEASURE -I Q samples 0 or 1 and collapses the state.
6. RETURNVALS Q declares Q as the first public result.



2.2 What one run means

One execution gives one random-looking sample. For this circuit, theory predicts equal probability for 0 and 1. Getting 0 once does not mean H failed, and getting 1 once does not prove a bias. Repeat the circuit to compare observed frequencies with the predicted distribution.

2.3 Adding user-controlled input

Move a wire into PARAMS when callers or truth-table rows should choose its start state:

```
PARAMS: Input:state

MAIN-PROCESS Inverter
X -I Input -0 Input
MEASURE -I Input
RETURNVALS Input
```

The matching deterministic truth table is:

Input	Output
0p	1p
1p	0p

The rest of this manual explains every symbol in these examples before moving to multi-qubit gates, child processes, and full truth-table files.

3 Lexical and file syntax

3.1 Logical lines, whitespace, and case

Commands and flags are case-insensitive and are separated by whitespace. Token names retain their spelling. CRLF and LF line endings are accepted. A physical line ending *exactly* in a backslash joins to the next line:

```
PARAMS: A:state B:state \
        Cin:state
CCNOT -I A B \
        -0 Carry
```

The continuation backslash must be the final character; spaces after it prevent continuation. Blank logical lines are discarded.

3.2 Hash comments

```
# Whole-line or trailing hash comment
H -I Q:0 -O Q:0 # prepare |+>
```

A hash starts a comment that continues through the logical line. Comments are for explanations and do not become instructions. Because continuation joining happens first, a hash on a continued logical line can hide text that appeared on a later physical line. Prefer comments after a complete instruction.

3.3 Block comments

```
/* A block comment may span logical lines. */
PHASE=pi/2 -I Q:0 -O Q:0
```

Text from `/*` through `*/` is ignored. A block may occupy one line or continue across several lines. Block comments are useful for temporarily annotating a region, but they should not be nested. Keep delimiters visually obvious so an omitted closing marker does not hide the remainder of a file.

3.4 PARAMS:: the process input declaration

```
PARAMS: A:state B:state Theta:float Count:int
```

`PARAMS:` is recognized only when it is the first non-comment logical line. Each whitespace-separated entry containing a colon becomes `name:type`. Declaration order is interface order: `RUNCHILD` inputs and truth-table input columns map from left to right.

Long declarations can use continuation:

```
PARAMS: A0:state A1:state B0:state B1:state \
        Cin:state Width:int
```

The system retains the type text:

- `state` entries become user-facing circuit inputs and truth-table input columns. Each participating state parameter resolves to a qubit wire.
- `int` and `float` entries are retained but excluded from user-facing qubit controls and truth-table state inputs. They describe numeric process metadata rather than prepared qubits.
- other type names are retained as written. Because they are not one of the recognized numeric exclusions, they may be exposed like stateful parameters; prefer `state`, `int`, or `float` for predictable behavior.

`PARAMS` has a colon after the keyword; `MAIN-PROCESS` does not. An empty or misplaced `PARAMS` line is not a substitute for a valid first-line declaration. The process's `MAIN-PROCESS` instruction is described in the composition chapter.

3.5 Protocol file forms

The normal authoring format is plain UTF-8 QPU text saved as `ProcessName.qpucir`. A `.qpucir.txt` download is also plain source and is useful where browsers or editors insist on a text extension. Uploading plain text derives the display name from `MAIN-PROCESS`.

The loader can also accept a JSON export envelope:

```
{
  "format": "qpucir",
  "version": 1,
  "name": "First circuit",
  "source": "MAIN-PROCESS FirstCircuit\nRETURNVALS Q"
}
```

The source string is the authoritative language text. Export envelopes may also contain compiled qubit count, gates, token map, and export timestamp metadata, but compilation from source determines current behavior.

4 Names and token references

A *token* is the QPU language’s name for a logical storage location. For the circuits in this guide, a token normally identifies one qubit wire. Instructions refer to tokens instead of physical machine addresses. During compilation, the system resolves each live token to an integer qubit index. This indirection makes protocols readable and lets child processes have private workspace without colliding with their parents.

4.1 Token names

A bare token is written as a whitespace-delimited name:

```
Q
Control
CarryOut
scratch_2
0
```

Names are case-sensitive even though instruction names are not: **Carry** and **carry** are distinct tokens. A name cannot contain whitespace because whitespace separates language elements. Avoid names that begin with a hyphen: inside a flag list, any hyphen-prefixed token is treated as the beginning of another flag.

Tokens are allocated lazily. Mentioning a token in a gate, a state-setting instruction, or **CREATETOKEN** gives it a wire when compilation needs one. Consequently, source order affects the internal numeric wire index, but not the meaning of a consistently named circuit.

4.2 Direct references

A direct reference is simply the token’s name:

```
SET Q Op
H -I Q -O Q
MEASURE -I Q
RETURNVALS Q
```

Every occurrence resolves to the same token within that process invocation. For a one-qubit gate, repeat the name after both **-I** and **-O**. This does not copy the state. It says that **Q** is the instruction’s incoming wire and that the same **Q** wire is mutated in place.

4.3 Parameter references with a dollar sign

A leading dollar sign is the explicit parameter/reference spelling:

```
PARAMS: Input:state
MAIN-PROCESS DollarReference
SET Local $Input
H -I $Input -O $Input
RETURNVALS Local
```

Address resolution removes the leading **\$**. Thus **Input** and **\$Input** ultimately refer to the same declared parameter. The dollar form is especially useful inside reusable child protocols because it visually separates caller-supplied values from child-local workspace.

SET Local \ \$Input creates an alias: **Local** resolves to the same underlying wire as **Input**. It does *not* clone an unknown quantum state. Quantum states cannot generally be copied because of the no-cloning theorem; aliasing avoids claiming that a physical copy occurred.

4.4 Cycle-qualified references with :

A token may carry a suffix after a colon:

```
Q:0
Q:1
Carry:next
```

The current system strips the colon and everything after it when mapping source references to visual qubit wires. Therefore all three examples above whose base is Q—Q:0, Q:1, and bare Q—identify the same logical wire. The suffix is protocol notation for a stage, cycle, or bit version; it does not create a second qubit.

```
SET Q:0 Op
H -I Q:0 -O Q:1
MEASURE -I Q:2
```

This example applies H to one wire and then measures that same wire. The changing suffix communicates progression to a human reader, while INCREASECYCLE provides the actual compiler cycle boundary.

Practical rule: if two values must exist simultaneously, give them different base names. Do not expect Q:0 and Q:1 to hold two independent states.

4.5 Aliases created by SET

When the second operand of SET is not a built-in state constant, SET creates a name alias:

```
SET Working Input
SET Output Working
X -I Output -O Output
```

After resolution, Working, Output, and Input name one wire. Mutating any alias mutates that wire for every alias. This behavior is useful for readable names and process binding, but it is not assignment in the classical “copy a value into a new variable” sense.

Aliases are resolved within a process frame. PARAMS and parent/child bindings take precedence over ordinary child-local scoping, ensuring that an operation on a bound parameter affects the caller’s intended wire.

4.6 Numeric workspace references

Bare numeric names such as 0, 1, and 2 are valid token names and are common in reusable arithmetic children:

```
SET 0:0 $A
SET 1:0 $B
SET 2:0 $Cin
```

When a child process uses a numeric workspace name, the compiler places it in the parent invocation’s workspace namespace. This lets nested adders reuse the same compact source names without globally identifying every child invocation’s scratch wires.

Numeric token names are not number literals. 2 names a wire; it does not mean the integer two or the basis state $|2\rangle$.

4.7 Process scope and child-local references

Each process invocation receives a unique scope. A local name such as **Scratch** in two separate child calls therefore denotes two separately resolved local references unless output or parameter binding deliberately joins them. Conceptually, compilation turns:

```
RUNCHILD Worker -I A -O First
RUNCHILD Worker -I B -O Second
```

into two independent Worker frames. This is similar to local variables in two function calls. PARAMS map caller references into the frame, and RETURNVALS map selected frame references back out.

5 Built-in state constants

State constants describe how a wire begins. They are not ordinary token names, even though they participate in address resolution. Constants are case-insensitive and may be written with a leading dollar sign. The three supported base constants are `0p`, `1p`, and `sp`; the letter `p` distinguishes protocol state notation from ordinary numbers.

5.1 0p: the zero basis state

`0p` denotes

$$|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix}.$$

SET Output `0p`

For workspace, this schedules zero preparation. The compiler batches pending zero preparations into internal RESET operations immediately before the next real gate, at an `INCREASECYCLE`, or at process end. RESET is deliberately not source syntax; **SET** Output `0p` is the public way to request zero.

Zero is the usual target initialization for reversible AND, OR, and XOR: with $t = 0$, the mutation $t \leftarrow t \oplus f(x)$ leaves $t = f(x)$.

5.2 1p: the one basis state

`1p` denotes

$$|1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

SET Flag `1p`

The compiler realizes one preparation by applying X to a zero-initialized wire. A target beginning in one is useful when the desired reversible result is the complement of a Boolean predicate. For example, $1 \oplus (A \wedge B) = \neg(A \wedge B)$, which is NAND.

5.3 sp: equal superposition

`sp` denotes the plus state

$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

SET Probe `sp`

The compiler realizes this preparation with a Hadamard gate. If immediately measured, the result is 0 or 1 with equal probability. It is not merely a hidden classical random bit: both amplitudes coexist and can interfere after later phase and Hadamard operations.

In an expected truth-table *output* cell, `sp` has a different testing role: it means either measured bit is acceptable. Context therefore matters—protocol SET initializes a plus state, while expected output metadata uses `sp` as a don't-care marker.

5.4 Dimension-suffixed constants

The language also recognizes:

```
Op_dim2
1p_dim8
sp_dim4
```

The suffix must be `_dim` followed by one or more decimal digits. These forms preserve compatibility with protocols that annotate a state constant's intended dimension. In the current simulator they still perform the same single-wire preparation as their unsuffixed forms; the dimension number does not allocate a multi-qubit register or change vector size.

5.5 Constants and state parameters

For an ordinary local token, SET emits preparation behavior. For a PARAMS entry typed `state`, SET supplies a default instead:

```
PARAMS: Input:state
MAIN-PROCESS DefaultInput
SET Input Op
X -I Input -O Input
RETURNVALS Input
```

The UI or truth-table runner may replace the default start state for Input. This distinction is what allows one compiled protocol to be tested over every binary input row instead of hard-wiring its parameter to zero.

6 Instruction flags and argument boundaries

6.1 -I: input and control references

`-I` begins the instruction's input list:

```
H      -I Target      -O Target
CNOT -I Control      -O Target
CCNOT -I A B          -O Target
```

Its interpretation depends on the gate. For a one-qubit gate, the input names the same wire that the output mutates. For CNOT/CZ/CY it names one control. For CCNOT and the two-input derived logic gates it names two controls. Input order does not matter to symmetric predicates such as AND or XOR, but it remains part of the written instruction and should be kept consistent for readability.

6.2 -O: output and target references

`-O` begins the output list. For most gates its first token is the target wire whose amplitudes are changed:

```
X -I Q -O Q
AND -I A B -O Result
```

`-O` does not mean “allocate a fresh classical output.” The target may already contain a quantum state, and reversible logic XORs its result into that existing state. SWAP is exceptional: it requires two outputs matching its two inputs. MEASURE takes no output flag because the measurement map stores its classical result.

6.3 Where a flag list ends

An `-I` or `-O` list continues until the next token beginning with a hyphen or the end of the line:

```
CCNOT -I A B -O Carry -$R
```

Here A and B belong to `-I`, Carry belongs to `-O`, and `-$R` starts a separate flag. This rule is why token names should not begin with a hyphen.

Flags are case-insensitive, so `-i` and `-o` work, but uppercase is the documented style. Supplying flags in an unusual order may still parse, but `-I` followed by `-O` is the clearest and matches generated source.

6.4 No-parameter-substitution marker

```
PHASE=pi/4 -I Q -O Q -$R
```

The language recognizes this marker and preserves it on the instruction record as a request not to substitute parameters. The current compiler does not yet change address resolution or gate execution because of the marker. It is therefore compatibility syntax, not a reliable way to freeze a reference. Write explicit token and parameter bindings instead.

6.5 Positional arguments

Not every instruction uses flags. `SET`, `MAIN-PROCESS`, `RETURNVALS`, `ACCEPTVALS`, and several compatibility instructions consume positional words:

```
SET Target Op
MAIN-PROCESS Example
RETURNVALS Carry Sum
```

The command name is the first word and positional arguments follow in order. `RETURNVALS` order is part of the process interface. For non-gate compatibility instructions, extra words may be retained even when no execution behavior is defined; accepted text should not be mistaken for an implemented effect.

7 Gate instruction grammar and arity

7.1 General gate shape

```
GATE -I input_or_controls -O target
```

The uppercase word selects a gate. `-I` lists the incoming wire for a one-qubit operation or the controls for a controlled operation. `-O` names the wire that is mutated. Except for `SWAP`, gate instructions do not create a separate output wire automatically.

```
H -I Q -O Q
CNOT -I Control -O Target
```

In the first row, `Q` is listed twice because the state flows through `H` on one wire. In the second, `Control` is preserved and `Target` is conditionally changed.

7.2 Why arity matters

Gate family	Required <code>-I</code> count	Required <code>-O</code> count
X, Y, Z, H, S, T, PHASE, NOT	1	1
CNOT, CZ, CY	1	1
CCNOT, AND, NAND, OR, XOR	2	1
SWAP	2	2
MEASURE	0 or 1	0

All counts are exact except `MEASURE`. For ordinary single-qubit gates, name the same wire in `-I` and `-O`. The output is the mutated target; for controlled gates, `-I` contains controls.

7.3 Validation and common mistakes

Arity is part of the language contract. A CNOT with two inputs is not silently converted into CCNOT, and a CCNOT with one input is not converted into CNOT. The compiler reports whether an instruction has too few or too many inputs or outputs.

```
# Wrong: H needs one output.
H -I Q

# Wrong: CNOT needs exactly one control.
CNOT -I A B -O Target

# Right:
CCNOT -I A B -O Target
```

The target is removed from the compiler's control list if the same resolved wire appears on both sides. This is necessary for the normal one-qubit form. For controlled gates, do not also list the target as an input; doing so either violates arity or obscures the intended control relationship.

8 Mathematical foundations

This chapter assumes no prior quantum mechanics. The essential ideas are numbers, vectors, probability, and transformations. Quantum notation can look unfamiliar, but each symbol has a concrete computational meaning.

8.1 Real numbers and complex numbers

The real numbers include familiar values such as -2 , 0 , $1/3$, and $\sqrt{2}$. A real number can be placed on a one-dimensional number line. Quantum amplitudes require a second numerical direction, represented by the *imaginary unit*

$$i = \sqrt{-1}, \quad i^2 = -1.$$

There is no real number whose square is -1 , so i extends the number system. A complex number has the form

$$z = a + bi,$$

where a is the real part and b is the imaginary part. Examples are $3 + 2i$, $-i$, and 1 (which is $1 + 0i$).

Complex arithmetic follows ordinary algebra together with $i^2 = -1$:

$$\begin{aligned}(a + bi) + (c + di) &= (a + c) + (b + d)i, \\ (a + bi)(c + di) &= (ac - bd) + (ad + bc)i.\end{aligned}$$

For example,

$$(1 + i)^2 = 1 + 2i + i^2 = 2i.$$

8.2 The complex plane, magnitude, and conjugate

A complex number is a point or arrow in a plane: horizontal coordinate a , vertical coordinate b . Its magnitude is the arrow length,

$$|z| = \sqrt{a^2 + b^2}.$$

The complex conjugate reflects the arrow across the real axis:

$$z^* = a - bi.$$

Multiplying by the conjugate gives a nonnegative real number:

$$z^*z = (a - bi)(a + bi) = a^2 + b^2 = |z|^2.$$

Quantum measurement probabilities use $|z|^2$, not z itself. This guarantees real, nonnegative probabilities even though amplitudes may be complex.

8.3 Angles, radians, sine, and cosine

An arrow in the complex plane may be described by its length r and angle θ :

$$a = r \cos \theta, \quad b = r \sin \theta.$$

Quantum formulas normally measure angles in radians. One full turn is 2π radians, a half turn is π , and a quarter turn is $\pi/2$. Conversion is

$$\theta_{\text{radians}} = \theta_{\text{degrees}} \frac{\pi}{180}.$$

This is why `PHASE=90d` and `PHASE=pi/2` describe the same angle.

8.4 Euler's formula and Euler's identity

Euler's formula connects complex numbers to rotations:

$$e^{i\theta} = \cos \theta + i \sin \theta.$$

The value $e^{i\theta}$ always has magnitude one. As θ changes, it moves around the unit circle in the complex plane. Important cases are

$$e^{i0} = 1, \quad e^{i\pi/2} = i, \quad e^{i\pi} = -1, \quad e^{i2\pi} = 1.$$

Setting $\theta = \pi$ gives Euler's identity,

$$e^{i\pi} + 1 = 0.$$

In a phase gate, multiplying an amplitude by $e^{i\theta}$ rotates that amplitude without changing its magnitude. Its immediate measurement probability is unchanged, but its relationship to other amplitudes changes. Later gates can convert that relative phase into different probabilities.

8.5 Vectors and basis vectors

A column vector is an ordered list of numbers. The two basis vectors used for one qubit are

$$|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad |1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

The vertical bars and angle bracket form Dirac's *ket* notation. `0p` prepares $|0\rangle$ and `1p` prepares $|1\rangle$. Any one-qubit pure state is a complex linear combination:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle = \begin{bmatrix} \alpha \\ \beta \end{bmatrix}.$$

The complex numbers α and β are *probability amplitudes*. They are not probabilities themselves.

8.6 Normalization and the Born rule

A valid state is normalized:

$$|\alpha|^2 + |\beta|^2 = 1.$$

The Born rule turns amplitudes into measurement probabilities:

$$P(0) = |\alpha|^2, \quad P(1) = |\beta|^2.$$

For the plus state,

$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}},$$

both amplitudes are $1/\sqrt{2}$. Squaring their magnitudes gives $P(0) = P(1) = 1/2$. The factor $1/\sqrt{2}$ is therefore required; writing $|0\rangle + |1\rangle$ without normalization would produce total probability two.

8.7 Matrices as gates

A matrix is a rectangular array of numbers. A one-qubit gate is represented by a 2×2 complex matrix. Matrix-vector multiplication computes the new amplitudes:

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} \alpha \\ \beta \end{bmatrix} = \begin{bmatrix} a\alpha + b\beta \\ c\alpha + d\beta \end{bmatrix}.$$

For example, the X gate exchanges the two amplitudes:

$$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} \alpha \\ \beta \end{bmatrix} = \begin{bmatrix} \beta \\ \alpha \end{bmatrix}.$$

Applying gates in source order means multiplying by matrices in the same temporal order: if U acts first and V second, then

$$|\psi_{\text{final}}\rangle = V(U|\psi_{\text{initial}}\rangle) = VU|\psi_{\text{initial}}\rangle.$$

8.8 Unitarity and reversibility

Closed-system quantum gates must preserve total probability. Their matrices are *unitary*:

$$U^\dagger U = I.$$

Here $U^\dagger$ means transpose the matrix and complex-conjugate every entry; I is the identity matrix. Unitarity guarantees an inverse, $U^{-1} = U^\dagger$. Quantum gates therefore cannot normally erase information. Classical functions such as AND are embedded reversibly by preserving inputs and XORing the result into a target:

$$|a, b, t\rangle \mapsto |a, b, t \oplus (a \wedge b)\rangle.$$

Measurement and reset are exceptions because they interact with classical outcomes or preparation rather than acting as closed unitary gates.

9 Qubits, phase, and the Bloch sphere

9.1 A qubit is not just an unknown bit

A classical bit is exactly 0 or 1. A qubit may be in any normalized superposition $\alpha|0\rangle + \beta|1\rangle$. This does not mean that the qubit secretly chose a classical value before measurement. Relative amplitudes can interfere, producing effects no ordinary random bit can reproduce.

9.2 Global phase and relative phase

Multiplying an entire state by a unit complex number does not change physical predictions:

$$|\psi\rangle \quad \text{and} \quad e^{i\gamma}|\psi\rangle$$

have the same measurement probabilities and are considered equivalent up to a *global phase*. Relative phase is different. The states

$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}, \quad |-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}$$

give equal 0/1 probabilities if measured immediately, yet H sends $|+\rangle \mapsto |0\rangle$ and $|-\rangle \mapsto |1\rangle$. Their relative sign is observable through interference.

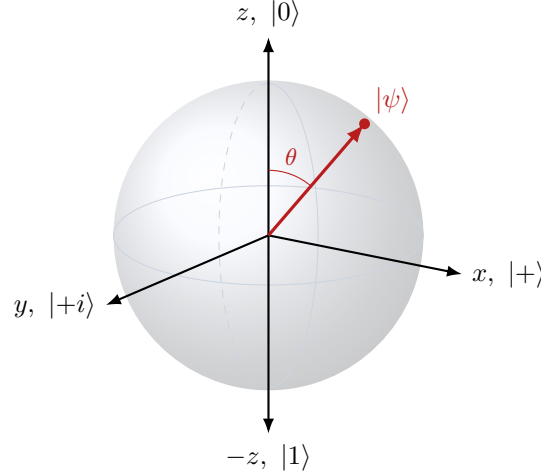
9.3 Bloch-sphere coordinates

After removing an unobservable global phase, every pure one-qubit state can be written with two real angles:

$$|\psi\rangle = \cos\left(\frac{\theta}{2}\right)|0\rangle + e^{i\phi}\sin\left(\frac{\theta}{2}\right)|1\rangle,$$

where $0 \leq \theta \leq \pi$ and $0 \leq \phi < 2\pi$. These angles define a point on the unit Bloch sphere:

$$x = \sin\theta \cos\phi, \quad y = \sin\theta \sin\phi, \quad z = \cos\theta.$$



The north pole is $|0\rangle$ and the south pole is $|1\rangle$. The equator contains equal-magnitude superpositions. $|+\rangle$ points along $+x$, $|-\rangle = (|0\rangle - |1\rangle)/\sqrt{2}$ along $-x$, and $|+i\rangle = (|0\rangle + i|1\rangle)/\sqrt{2}$ along $+y$.

9.4 How gates move a Bloch vector

Ignoring global phase, many one-qubit gates are rotations:

- X rotates the sphere by π around the x-axis, exchanging north and south.
- Y rotates by π around the y-axis.
- Z rotates by π around the z-axis, changing relative phase.
- S and T rotate around z by $\pi/2$ and $\pi/4$.
- PHASE rotates around z by its supplied angle θ .
- H rotates the coordinate description so the z basis and x basis are exchanged: $|0\rangle \leftrightarrow |+\rangle$ and $|1\rangle \leftrightarrow |-\rangle$.

The sphere is a teaching model for one pure qubit. It cannot display the full state of entangled multi-qubit systems as one ordinary three-dimensional arrow.

10 Multiple qubits and entanglement

10.1 Tensor products

Two classical bits have four possibilities; two qubits have four basis vectors:

$$|00\rangle, |01\rangle, |10\rangle, |11\rangle.$$

Independent systems combine with the tensor product $\otimes$. If $|A\rangle = [a_0, a_1]^T$ and $|B\rangle = [b_0, b_1]^T$, then

$$|A\rangle \otimes |B\rangle = \begin{bmatrix} a_0 b_0 \\ a_0 b_1 \\ a_1 b_0 \\ a_1 b_1 \end{bmatrix}.$$

For n qubits, the simulator stores 2^n amplitudes:

$$|\Psi\rangle = \sum_{x=0}^{2^n-1} a_x |x\rangle, \quad \sum_x |a_x|^2 = 1.$$

This exponential growth is why state-vector simulation becomes expensive as qubit count increases.

10.2 Entanglement

Some multi-qubit states cannot be factored into independent one-qubit states. The Bell state

$$|\Phi^+\rangle = \frac{|00\rangle + |11\rangle}{\sqrt{2}}$$

is entangled. Each qubit measured alone is equally likely to be 0 or 1, but their outcomes are perfectly correlated. It can be prepared from $|00\rangle$:

```
H -I A -0 A
CNOT -I A -0 B
```

The first gate creates a superposition; the controlled gate correlates B with A. Entanglement is not a classical alias. It is a property of the joint amplitude vector.

10.3 Controlled operations

A controlled gate applies a target operation only on basis components where its control condition is true. CNOT applies X when its control is one. If the control is in superposition, linearity applies the operation to one branch and not the other, which can create entanglement:

$$\frac{|0\rangle + |1\rangle}{\sqrt{2}} \otimes |0\rangle \xrightarrow{\text{CNOT}} \frac{|00\rangle + |11\rangle}{\sqrt{2}}.$$

CCNOT requires both controls to be one. The system's OR and XOR derived gates use “any active” and “odd parity” predicates respectively.

11 Measurement, probability, and truth tables

11.1 Projective measurement

For a measured qubit q in a multi-qubit state,

$$P(q = 1) = \sum_{x: x_q=1} |a_x|^2, \quad P(q = 0) = 1 - P(q = 1).$$

The simulator samples one outcome. Amplitudes incompatible with that outcome become zero; compatible amplitudes are divided by the square root of the kept probability so the state is normalized again. Measurement is destructive in the sense that the pre-measurement superposition is generally unavailable after collapse.

11.2 Single runs and repeated runs

One simulation run produces one measurement sample. To estimate a probability, run the same preparation and circuit many times. If a plus state is measured 1 approximately 500 times in 1000 shots, the observed frequency 500/1000 estimates the theoretical probability 1/2. More shots reduce ordinary sampling noise but never reveal a complex amplitude directly.

11.3 Truth tables versus quantum amplitudes

A classical truth table specifies output bits for listed input bits. It is a good contract for arithmetic and reversible Boolean circuits operating on computational-basis states. It cannot fully describe phase, interference, entanglement, or a probability distribution.

The QPU truth-table runner prepares each input row, executes the circuit, measures declared RETURNVALS, and compares those measured bits. In companion .qpuio tables, cells are `0p`, `1p`, or `sp`. On an expected output, `sp` means “either measured bit is acceptable.” Use state equations and repeated experiments in addition to tables when a circuit’s purpose is genuinely quantum.

11.4 A complete interference example

Starting in zero, two Hadamards restore zero:

$$|0\rangle \xrightarrow{H} |+\rangle \xrightarrow{H} |0\rangle.$$

Inserting Z changes the relative phase and changes the final answer:

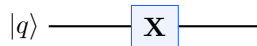
$$|0\rangle \xrightarrow{H} |+\rangle \xrightarrow{Z} |-\rangle \xrightarrow{H} |1\rangle.$$

```
MAIN-PROCESS Interference
CREATETOKEN -I Q
SET Q 0p
H -I Q -0 Q
Z -I Q -0 Q
H -I Q -0 Q
MEASURE -I Q
RETURNVALS Q
```

The middle Z does not change an immediate 0/1 probability, yet interference at the final H makes its effect deterministic. This is the central reason the system documents matrices and phase as well as truth tables.

12 Single-qubit primitive gates

12.1 Pauli X (X)



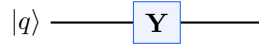
$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad X|0\rangle = |1\rangle, \quad X|1\rangle = |0\rangle, \quad X(\alpha|0\rangle + \beta|1\rangle) = \beta|0\rangle + \alpha|1\rangle.$$

X is the quantum bit flip. On the Bloch sphere it is a half-turn around the x-axis. It exchanges the north and south poles and is self-inverse: $X^2 = I$. Unlike a classical NOT implementation, X remains meaningful on a superposition because it exchanges amplitudes linearly without measuring.

```
X -I Q:0 -0 Q:0
```

Input	Classical comparison	Output
0	NOT 0	1
1	NOT 1	0

12.2 Pauli Y (Y)



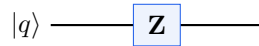
$$Y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}, \quad Y|0\rangle = i|1\rangle, \quad Y|1\rangle = -i|0\rangle.$$

It flips the basis value like X but adds phases that become observable through later interference. Geometrically it is a half-turn around the y-axis and is self-inverse, $Y^2 = I$. The factors i and $-i$ are global phases for either basis state by itself, but on components of a larger superposition they may become relative phases.

```
Y -I Q -0 Q
```

Input	Measured bit	Quantum output
0	1	$i 1\rangle$
1	0	$-i 0\rangle$

12.3 Pauli Z (Z)



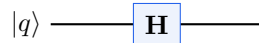
$$Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}, \quad Z|0\rangle = |0\rangle, \quad Z|1\rangle = -|1\rangle.$$

Z does not change a computational-basis measurement, but it changes relative phase. For example, $Z|+\rangle = |-\rangle = (|0\rangle - |1\rangle)/\sqrt{2}$. On the Bloch sphere it is a half-turn around z. It is self-inverse and is the simplest example showing why quantum state cannot be reduced to a table of immediate measurement probabilities.

```
Z -I Q -0 Q
```

Input	Classical bit	Quantum output
0	0	$ 0\rangle$
1	1	$- 1\rangle$

12.4 Hadamard (H)



$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, \quad H|0\rangle = |+\rangle, \quad H|1\rangle = |-\rangle.$$

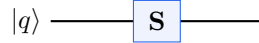
Hadamard changes between the computational z basis and the plus/minus x basis. It creates equal magnitudes from a basis state, but the sign of the second amplitude depends on the input. It is also self-inverse: applying H twice returns the original state. H is commonly used before controlled gates to create entanglement and after phase gates to reveal interference.

```
SET Q 0p
H -I Q -0 Q
MEASURE -I Q
```

Input	Measurement distribution	State before measurement
0	50% 0, 50% 1	$(0\rangle + 1\rangle)/\sqrt{2}$
1	50% 0, 50% 1	$(0\rangle - 1\rangle)/\sqrt{2}$

Equal classical probabilities hide the relative minus sign responsible for interference.

12.5 S phase (S)



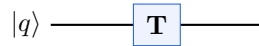
$$S = P(\pi/2) = \begin{bmatrix} 1 & 0 \\ 0 & i \end{bmatrix}, \quad S|1\rangle = i|1\rangle, \quad S^2 = Z.$$

S is a quarter-turn around the Bloch sphere's z-axis. It changes $|+\rangle$ into $(|0\rangle + i|1\rangle)/\sqrt{2}$ while leaving basis measurement probabilities unchanged. Its inverse is $S^\dagger = P(-\pi/2)$, not S itself.

S -I Q -0 Q

Input	Measured bit	Quantum output
0	0	$ 0\rangle$
1	1	$i 1\rangle$

12.6 T phase (T)



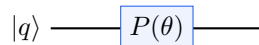
$$T = P(\pi/4) = \begin{bmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{bmatrix}, \quad T^2 = S, \quad T^4 = Z.$$

T is an eighth-turn around z. Together with gates such as H and CNOT, it is an important non-Clifford resource for expressing general quantum computations. Its inverse is $T^\dagger = P(-\pi/4)$; the current BT spelling does not synthesize that inverse, so use BPHASE=pi/4 when the negative phase is required.

T -I Q -0 Q

Input	Measured bit	Quantum output
0	0	$ 0\rangle$
1	1	$e^{i\pi/4} 1\rangle$

12.7 Arbitrary phase (PHASE)



$$P(\theta) = \begin{bmatrix} 1 & 0 \\ 0 & e^{i\theta} \end{bmatrix}, \quad \alpha|0\rangle + \beta|1\rangle \mapsto \alpha|0\rangle + \beta e^{i\theta}|1\rangle.$$

PHASE is the general form behind S, T, and Z. It preserves $|\alpha|^2$ and $|\beta|^2$ because $|e^{i\theta}| = 1$, yet rotates the relative phase by θ . On the Bloch sphere this is a z-axis rotation up to global phase. A later H or another basis-changing gate is normally needed to turn that phase difference into changed computational-basis probabilities. The angle is embedded in the opcode:

```

PHASE=1.5708 -I Q -0 Q      # radians
PHASE=3/2 -I Q -0 Q        # rational radians
PHASE=90d -I Q -0 Q        # degrees
PHASE=45/2d -I Q -0 Q      # rational degrees
PHASE=pi -I Q -0 Q
PHASE=-11pi/6 -I Q -0 Q
PHASE=2*pi/3 -I Q -0 Q
BPHASE=pi/4 -I Q -0 Q      # P(-pi/4)

```

Accepted numeric atoms include signed integers, decimals, leading-dot decimals, and one optional rational denominator. Pi notation accepts an optional sign, an integer coefficient, an optional *, and an optional positive integer denominator. A zero denominator and malformed angle raise an error.

Input	Measured bit	Quantum output
0	0	$ 0\rangle$
1	1	$e^{i\theta} 1\rangle$

12.8 Backward-prefix syntax

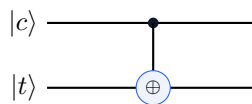
The QPU language recognizes a leading B before a supported primitive gate name, for example BX, BH, BCNOT, or BSWAP, and sets `reverse=true`. (Compiler-internal RESET remains unavailable in source.) Only `BPHASE=angle` currently changes emitted behavior: the compiler negates the phase angle. For every other gate the flag is metadata only. Emitting the ordinary gate still gives the mathematical inverse for these self-inverse gates:

X , Y , Z , H , CNOT, CCNOT, CZ, CY, SWAP.

In contrast, BS and BT do not synthesize $S^\dagger$ or $T^\dagger$. Do not rely on B-prefixed forms for inverse execution unless the gate is self-inverse or is BPHASE.

13 Controlled and two-qubit primitive gates

13.1 Controlled NOT (CNOT)



$$|c, t\rangle \mapsto |c, t \oplus c\rangle.$$

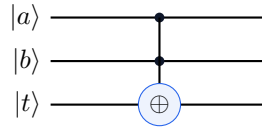
CNOT leaves the control unchanged. When the control is zero, the target is unchanged; when the control is one, X acts on the target. Because the map is a permutation of basis states, it is reversible and self-inverse. A superposition control can entangle the wires, so “classical XOR” describes only its basis-state action.

```
CNOT -I Control -0 Target
```

c	t	c'	t'
0	0	0	0
0	1	0	1
1	0	1	1
1	1	1	0

CNOT is a reversible embedding of XOR because it preserves the control.

13.2 Doubly controlled NOT / Toffoli (CCNOT)



$$|a, b, t\rangle \mapsto |a, b, t \oplus (a \wedge b)\rangle.$$

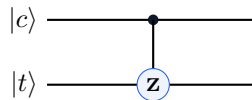
CCNOT, also called Toffoli, applies X only when both controls are one. It is a reversible universal building block for classical computation embedded in a quantum circuit. The target's previous value matters: zero computes AND, while one computes NAND. The gate is self-inverse.

CCNOT -I A B -0 Target

a	b	t	a'	b'	t'
0	0	0	0	0	0
0	0	1	0	0	1
0	1	0	0	1	0
0	1	1	0	1	1
1	0	0	1	0	0
1	0	1	1	0	1
1	1	0	1	1	1
1	1	1	1	1	0

With a target initialized to zero, Toffoli computes classical AND reversibly.

13.3 Controlled Z (CZ)



$$\text{CZ}|c, t\rangle = (-1)^{ct}|c, t\rangle, \quad \text{CZ} = \text{diag}(1, 1, 1, -1).$$

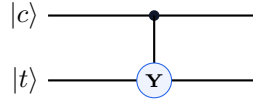
CZ adds a minus sign only to the $|11\rangle$ amplitude. No computational-basis bit changes, but superpositions can acquire observable relative phase. CZ is symmetric between its two wires, even though QPU syntax writes one as the input control and one as the output target. It is self-inverse.

CZ -I Control -0 Target

Input	Measured output	Amplitude multiplier	State
00	00	+1	$ 00\rangle$
01	01	+1	$ 01\rangle$
10	10	+1	$ 10\rangle$
11	11	-1	$- 11\rangle$

The classical columns are unchanged; the quantum phase is the operation.

13.4 Controlled Y (CY)



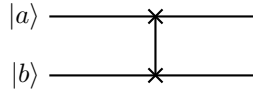
$$|0, t\rangle \mapsto |0, t\rangle, \quad |1, 0\rangle \mapsto i|1, 1\rangle, \quad |1, 1\rangle \mapsto -i|1, 0\rangle.$$

CY applies the Pauli Y rotation to the target only when the control is one. Its basis-bit pattern resembles CNOT, but the $\pm i$ factors distinguish their behavior on superpositions. Since $Y^2 = I$, CY is self-inverse.

CY -I Control -0 Target

c	t	c'	t'	Quantum phase
0	0	0	0	+1
0	1	0	1	+1
1	0	1	1	i
1	1	1	0	$-i$

13.5 Swap (SWAP)



$$\text{SWAP}|a, b\rangle = |b, a\rangle.$$

SWAP exchanges the complete quantum states of two wires, including amplitudes and any correlations with the rest of the register. It does not copy either state. Three CNOTs can decompose SWAP, but the QPU system provides it as one primitive instruction. Applying SWAP twice restores the original ordering.

SWAP -I A B -0 A B

The two output tokens are mandatory and must resolve to the same two wires, in the same order, as the inputs.

a	b	a'	b'
0	0	0	0
0	1	1	0
1	0	0	1
1	1	1	1

14 Derived logical gates

Derived gates are QPU instructions implemented as reversible target mutations. Inputs are preserved. If target t starts in zero, AND/OR/XOR write the familiar classical function. If it does not, the true behavior is the XOR embedding shown below.

14.1 Logical NOT (NOT)



$$t' = t \oplus 1.$$

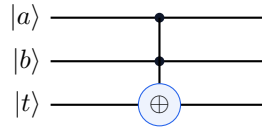
NOT is the derived logical spelling of a bit flip. In the current simulator it uses the same X matrix on the output target. The required input establishes the one-input instruction shape, but a distinct input is not copied into the output; use the same token on both sides.

NOT -I Target -O Target

t	t'
0	1
1	0

The current gate implementation mutates the output target. Use the same token for input and output; a distinct input token is not copied.

14.2 Reversible AND (AND)



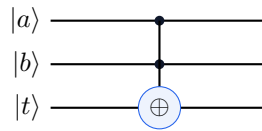
$$t' = t \oplus (a \wedge b).$$

AND flips the target exactly on the basis components where both controls are one. This is Toffoli-style behavior. It remains linear on superpositions and may entangle the target with its controls. Calling it AND is convenient only when the target is deliberately initialized.

SET Out Op
AND -I A B -O Out

a	b	$t = 0$	$t' = a \wedge b$
0	0	0	0
0	1	0	0
1	0	0	0
1	1	0	1

14.3 Reversible NAND (NAND)



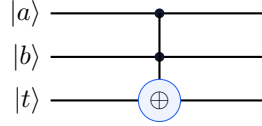
$$t' = t \oplus (a \wedge b) \oplus 1.$$

NAND first performs the all-controls-active mutation and then inverts the target. With target zero it produces the familiar NAND column. With target one it instead produces AND, illustrating why reversible target state must always be included in the full mathematical definition.

SET Out Op
NAND -I A B -O Out

a	b	$t = 0$	$t' = \neg(a \wedge b)$
0	0	0	1
0	1	0	1
1	0	0	1
1	1	0	0

14.4 Reversible OR (OR)



$$t' = t \oplus (a \vee b).$$

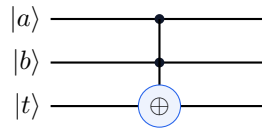
OR uses an “any control is one” predicate. If either or both controls are active, the target is flipped. This direct derived operation is equivalent on basis states to combining XOR and AND through $a \vee b = a \oplus b \oplus (a \wedge b)$.

```
SET Out Op
OR -I A B -O Out
```

a	b	$t = 0$	$t' = a \vee b$
0	0	0	0
0	1	0	1
1	0	0	1
1	1	0	1

For bits, $a \vee b = a \oplus b \oplus (a \wedge b)$.

14.5 Reversible XOR (XOR)



$$t' = t \oplus a \oplus b.$$

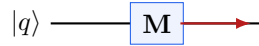
XOR flips the target when an odd number of controls is one. With two inputs, this is the ordinary exclusive-or predicate. The parity definition explains why the 11 row does not flip: two active controls have even parity.

```
SET Out Op
XOR -I A B -O Out
```

a	b	$t = 0$	$t' = a \oplus b$
0	0	0	0
0	1	0	1
1	0	0	1
1	1	0	0

15 Measurement gate

15.1 Measurement (MEASURE)



```
MEASURE -I Q      # exactly one named wire
MEASURE           # every wire known at this point
```

MEASURE takes no `-O`. Repeated one-wire commands are useful when only declared returns should become classical. A bare command emits one measurement operation per currently allocated wire.

State	Outcome probabilities	Collapsed state
$ 0\rangle$	$P(0) = 1$	$ 0\rangle$
$ 1\rangle$	$P(1) = 1$	$ 1\rangle$
$\alpha 0\rangle + \beta 1\rangle$	$P(0) = \alpha ^2, P(1) = \beta ^2$	$ 0\rangle$ or $ 1\rangle$

16 State, token, and cycle commands

16.1 SET — initialize or alias (*lowered/executed*)

```
SET Q Op
SET Q 1p
SET Q sp
SET Local $Parameter
```

SET **target value** requires a positional value. A constant schedules or emits initialization: zero, X from zero, or H from zero. A nonconstant value makes the target an alias of that source. A **state** parameter's constant SET is treated as a runtime default, allowing the UI or truth-table runner to supply its actual start state.

Syntax: exactly one target and one value are meaningful. SET has no `-I` or `-O` flags. It does not perform classical assignment at each simulation step; it establishes compilation-time preparation or reference identity. See the dedicated constants and aliases chapters before using SET on unknown quantum data.

16.2 CREATETOKEN — reserve wires (*lowered/executed*)

```
CREATETOKEN -I Sum Carry Workspace
```

Every `-I` token immediately claims a stable wire index. No `-O` is used. This is useful for declared outputs before their first gate.

CREATETOKEN does not initialize amplitudes by itself. Pair new workspace with SET before relying on its value. Listing outputs early also makes process structure easier to review:

```
CREATETOKEN -I Sum Carry
SET Sum Op
SET Carry Op
```

16.3 DELETETOKEN — token lifetime marker (*accepted, not executed*)

```
DELETETOKEN -I Workspace
```

This says that the named token is no longer needed by the protocol. The current compiler records the instruction but does not remove, reset, or reuse its wire. It is unsafe to assume that deletion destroys sensitive state or makes a qubit

available. Unused symbolic holes may disappear during final layout compaction, which is a separate implementation optimization.

16.4 FREE — releasable-memory marker (accepted, not executed)

```
FREE -I ScratchA ScratchB
```

FREE marks one or more input references as releasable. Like DELETETOKEN, it is currently a compatibility/lifetime hint: no physical reset, deallocation, or wire reuse occurs. Use SET with 0p when the intended behavior is actual zero preparation for subsequent computation.

16.5 INCREASECYCLE — cycle boundary (lowered/executed)

```
INCREASECYCLE
```

The compiler flushes pending zero-initializations into an internal batched RESET, increments the cycle counter, and prepares workspace for following operations. Cycle suffixes on tokens do not create separate visual wires.

Cycles organize protocol stages. They are not wall-clock timing guarantees and do not cause gates to execute concurrently. The final circuit is still an ordered gate sequence. Their practical implemented effect today is the workspace-zero boundary.

16.6 JOIN — composite-register marker (accepted, not executed)

```
JOIN -I A B -O AB
```

JOIN expresses the intent to view smaller registers as one composite register. For independent inputs, the mathematical construction is

$$|AB\rangle = |A\rangle \otimes |B\rangle.$$

Tensor product increases dimension multiplicatively: two one-qubit vectors of length two produce one two-qubit vector of length four. The current compiler logs JOIN but emits no tensor-product allocation, alias, or visible gate. Because JOIN is not a gate instruction, its input/output count is not currently validated.

16.7 SPLIT — component-register marker (accepted, not executed)

```
SPLIT AB A 2
```

SPLIT expresses the reverse organizational intent: identify a component of a larger register. A general entangled state cannot be factored into independent pure states, so “splitting” is not equivalent to slicing an ordinary array. For example, $(|00\rangle + |11\rangle)/\sqrt{2}$ has no separate pure state for A. The current compiler logs the positional arguments but performs no extraction, measurement, or aliasing. Code requiring an implemented result should instead operate directly on named wires and RETURNVALS.

17 Process composition and return values

17.1 MAIN-PROCESS (lowered/executed)

```
MAIN-PROCESS FullAdder
```

This marks the process body and supplies its catalog/library name. Compilation already begins from the named process, so the row itself emits no gate. The name is the first positional word after MAIN-PROCESS; avoid spaces inside it. This name connects a protocol to catalog entries, child calls, downloads, and companion truth-table metadata. If no marker exists, the system uses InlineProcess, but named processes are strongly recommended.

17.2 DECLARECHILD (accepted, not executed)

```
DECLARECHILD SingleBitFullAdder
```

The declaration is logged. It does not load inline source. Child bodies must exist in the external `librarySources` catalog when called. `DECLARECHILD` is therefore documentation/compatibility syntax rather than an import statement. A successful declaration does not guarantee that a later call can resolve. The process catalog supplied to compilation remains the source of truth.

17.3 RUNCHILD — expand a child process (lowered/executed)

```
RUNCHILD SingleBitFullAdder \
  -I A B Cin \
  -O Carry Sum
```

The first positional argument is the child process name. Inputs bind by the child’s `PARAMS` order: `A` maps to the first parameter, `B` to the second, and `Cin` to the third in this example. Outputs bind by the child’s `RETURNVALS` order. Before expansion, each bound parent output is zero-prepared once. Child-local names receive a unique scope. `RUNCHILD` is compile-time composition, not a dynamic subroutine call in a running quantum machine. The child body is recursively expanded into the parent’s flat gate sequence. This allows the renderer and simulator to inspect every resulting gate.

17.4 CALL — alternate child invocation (lowered/executed)

```
CALL SingleBitFullAdder -I A B Cin -O Carry Sum
```

`CALL` currently follows the same child lookup, parameter binding, output preparation, and expansion path as `RUNCHILD`. `RUNCHILD` is used by the bundled multi-bit protocols and makes compile-time expansion explicit; `CALL` is retained as an equivalent system spelling. Both fail when the named child is absent from the supplied process library.

17.5 RETURNVALS (lowered/executed)

```
RETURNVALS Carry Sum
```

Positional tokens define the process’s ordered public outputs. They determine child output binding, truth-table output columns, and the UI’s preferred logical width. Do not use `-I` or `-O` here: flags would be retained as positional return tokens.

`RETURNVALS` does not measure automatically. Add `MEASURE` instructions when classical outcomes are required. Returning an unmeasured wire preserves it as part of the projected quantum output shown by the system.

17.6 ACCEPTVALS (lowered/executed)

```
CALL Child -I A B
ACCEPTVALS LocalCarry LocalSum
```

Each positional local token aliases the corresponding value from the most recent child return list. Direct `-O` binding on `RUNCHILD`/`CALL` is clearer when output names are known.

Ordering is again essential: the first `ACCEPTVALS` name receives the first returned reference, and so on. The instruction refers to the most recently completed child expansion in compiler state; inserting another child call changes which return list is accepted.

17.7 Nested full-adder example

```

# child.qpucir
PARAMS: A:state B:state Cin:state
MAIN-PROCESS ChildAdder
CREATETOKEN -I Carry Sum
SET Carry Op
SET Sum Op
XOR -I A B -O Sum
CNOT -I Cin -O Sum
CCNOT -I A B -O Carry
CCNOT -I A Cin -O Carry
CCNOT -I B Cin -O Carry
RETURNVALS Carry Sum

# parent.qpucir
PARAMS: A:state B:state Cin:state
MAIN-PROCESS Parent
CREATETOKEN -I Cout Result
RUNCHILD ChildAdder -I A B Cin -O Cout Result
MEASURE -I Cout
MEASURE -I Result
RETURNVALS Cout Result

```

For basis inputs, the full-adder equations are

$$\text{Sum} = A \oplus B \oplus C_{\text{in}}, \quad C_{\text{out}} = (A \wedge B) \oplus (A \wedge C_{\text{in}}) \oplus (B \wedge C_{\text{in}}).$$

A	B	C_{in}	C_{out}	Sum
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

18 Accepted compatibility commands

These instructions are part of the accepted QPU language, but their present effect is to preserve arguments and add a compiler-log entry. Their acceptance must not be confused with simulator behavior.

18.1 COMPILEPROCESS (accepted, not executed)

```
COMPILEPROCESS ChildName
```

COMPILEPROCESS records a request associated with another process. It does not currently locate, compile, or insert that process. Use RUNCHILD or CALL with a process available in the compilation library when actual child gates are required. Arguments are positional and no arity is enforced.

18.2 MASTERVAL (accepted, not executed)

```
MASTERVAL Result
```

MASTERVAL records the intent that a reference is a final or authoritative value. It does not change the process interface, measurement behavior, token mapping, or RETURNVALS. Use RETURNVALS to expose outputs that the current system will actually project and associate with truth-table columns.

18.3 Save-state checkpoint instruction (accepted, not executed)

```
SAVE_STATE checkpointA
```

SAVE_STATE records a named checkpoint request. The simulator does not currently copy the state vector into a checkpoint store. In particular, later gates and measurements proceed exactly as if this line were absent. A log entry is not evidence that a recoverable snapshot exists.

18.4 Load-state checkpoint instruction (accepted, not executed)

```
LOAD_STATE checkpointA
```

LOAD_STATE records a restore request but does not replace the active state vector. It neither validates that a matching SAVE_STATE appeared nor rewinds measurements. Protocols whose correctness depends on restoration must wait for runtime support rather than relying on accepted syntax.

19 Truth-table companion syntax

Truth tables are metadata paired to a process; they are not `.qpucir` commands. A protocol describes *how* a circuit works; its companion `.qpuio` describes selected input/output expectations.

19.1 Text `.qpuio` structure

A text table may use spaces or commas:

```
MAIN-PROCESS: HalfAdder
INPUTS: A B
OUTPUTS: Carry Sum
# A B Carry Sum
0 0p 0p 0p 0p
1 0p 1p 0p 1p
2 1p 0p 0p 1p
3 1p 1p 1p 0p
```

19.2 MAIN-PROCESS: table header

The colon is required in the table format:

```
MAIN-PROCESS: HalfAdder
```

This name associates metadata with its protocol. Notice the deliberate syntax difference: protocol source writes `MAIN-PROCESS HalfAdder`, while the truth-table header writes `MAIN-PROCESS: HalfAdder`.

19.3 INPUTS: and OUTPUTS: column groups

```
INPUTS: A B
OUTPUTS: Carry Sum
```

These lines divide the columns into values supplied before simulation and values checked after simulation. Their order must match the later column header. INPUTS should correspond to state-typed PARAMS; OUTPUTS should correspond to RETURNVALS. OUTPUTS must contain at least one name.

Explicit groups remove ambiguity. If they are absent, the loader can derive the split only when a matching protocol is available and the combined PARAMS and RETURNVALS widths match the table header.

19.4 Column header row

```
# A B Carry Sum
```

The first nonmetadata row beginning with a hash and whitespace or comma is the ordered column header. Here A and B are inputs, followed by Carry and Sum outputs. Unlike an ordinary protocol hash comment, this row has structural meaning inside a .qpuio file.

19.5 Indexed data rows

```
0 0p 0p 0p 0p
1 0p 1p 0p 1p
```

The first cell is a decimal row index. Indices begin at zero and increase by one without gaps. Remaining cells align left-to-right with the column header. Every row must have exactly the expected width, and every state cell must be 0p, 1p, or sp.

For n binary input columns, a complete table contains 2^n rows. The usual ordering interprets input columns most-significant first: for two inputs the sequence is 00, 01, 10, 11. The loader checks sequential row indices, while semantic validation also rejects duplicate input patterns.

19.6 CSV spelling

Commas may replace whitespace consistently:

```
MAIN-PROCESS: HalfAdder
INPUTS: A,B
OUTPUTS: Carry,Sum
# ,A,B,Carry,Sum
0,0p,0p,0p,0p
```

Spaces around commas are trimmed. Do not mix extra prose into a data row; trailing hash comments are allowed only where the table loader can strip them after the data content.

19.7 Partial truth tables

A partial table lists selected cases rather than all 2^n combinations. It must contain at least one row, no duplicate input pattern, and no more than 2^n rows. Partial tables are useful for state-machine transitions or focused regressions, but passing one proves behavior only for the listed cases.

Input count n	Complete row count 2^n	Example
0	1	constant-output process
1	2	0, 1
2	4	00, 01, 10, 11
3	8	000 through 111
4	16	0000 through 1111

19.8 JSON .qpuio envelope

The equivalent machine-oriented JSON form is:

```
{
  "format": "qpuio",
  "version": 1,
  "processName": "HalfAdder",
  "inputColumns": ["A", "B"],
  "outputColumns": ["Carry", "Sum"],
  "rows": [
    ["0p", "0p", "0p", "0p"],
    ["0p", "1p", "0p", "1p"],
    ["1p", "0p", "0p", "1p"],
    ["1p", "1p", "1p", "0p"]
  ]
}
```

The `format` value must be `qpuio` and `version` must be 1. Column names must be nonempty strings, `outputColumns` must not be empty, and every nested row array must have exactly

`inputColumns.length + outputColumns.length`

valid cells.

19.9 How a table test runs

For each listed row, the runner:

1. maps input cells to state PARAMS by column name and order;
2. creates the circuit's start-state vector;
3. executes every compiled gate, including child expansions;
4. measures the declared RETURNVALS;
5. compares only those measured outputs with expected output cells.

An expected output `sp` accepts either measured bit. An input `sp` initializes an equal plus-state superposition. Because one probabilistic sample cannot establish a full distribution, tables are strongest for deterministic basis-state behavior.

19.10 Protected bundled tables

The bundled `SingleBitFullAdder`, `TwoBitFullAdder`, `FourBitFullAdder`, and `PhaseDemo` tables are canonical site assets. Do not rewrite them to make an experiment pass. Create a new uploaded or corrected process/table pair when testing different expectations. This keeps examples reproducible and prevents a protocol correction from silently changing its specification.

20 Complete worked protocol

```
PARAMS: A:state B:state Theta:float

MAIN-PROCESS BellAndLogic
CREATETOKEN -I Q0 Q1 AndOut

SET Q0 $A
SET Q1 $B
SET AndOut Op

# Quantum segment
H -I Q0 -O Q0
CNOT -I Q0 -O Q1
PHASE=pi/4 -I Q1 -O Q1
```

```
# Basis-state comparison output
AND -I A B -O AndOut

MEASURE -I Q0
MEASURE -I Q1
MEASURE -I AndOut
RETURNVALS Q0 Q1 AndOut
```

Starting from $|00\rangle$, the H-CNOT segment prepares

$$|00\rangle \xrightarrow{H \otimes I} \frac{|00\rangle + |10\rangle}{\sqrt{2}} \xrightarrow{\text{CNOT}} \frac{|00\rangle + |11\rangle}{\sqrt{2}}.$$

The phase gate then produces

$$\frac{|00\rangle + e^{i\pi/4}|11\rangle}{\sqrt{2}}.$$

The two quantum measurements are correlated, while `AndOut` compares the basis inputs through $A \wedge B$. A deterministic truth table can specify the logic output, but repeated simulation is needed to characterize Bell-state measurement probabilities.

21 Compiler output and lowering details

Compilation returns:

```
CompileResult {
  gates: CircuitGate[]
  qubitCount: number
  logicalQubitCount: number
  parsed: ParsedCommand[]
  log: string[]
  tokenMap: Record<string, number>
  processParams: ProcessParam[]
  returnValues: ReturnValue[]
}
```

Each accepted instruction entry records `op`, original `raw` text, `inputs`, `outputs`, positional `args`, optional normalized radian `phase`, `reverse`, and `noParameterSubstitution`. Each emitted gate records `id`, `type`, ordered `step`, `targets`, `controls`, optional `phase`, and source text.

After child expansion, unused symbolic holes are compacted to dense qubit indices. Internal RESET gates count for simulation but are filtered from the visible circuit. UI logical width prefers RETURNVALS count, then exposed PARAMS count, then physical simulator width.

22 Error conditions and practical rules

1. Unknown opcodes fail with `Unknown command`. RESET is not a source opcode.
2. Every QPU gate except MEASURE requires `-I` and `-O`, then exact registry arity is checked.
3. PHASE requires a valid nonzero-denominator angle after `=`.
4. SWAP requires two matching input/output wires.
5. RUNCHILD/CALL fails if the named process is absent from the library.
6. SET fails when its positional value is missing.
7. RETURNVALS order is semantically significant for child bindings and truth-table columns.
8. Use the same input/output token for single-qubit gates.
9. Initialize derived-gate targets deliberately; their operation is XOR into the existing target, not destructive assignment.
10. Treat accepted-but-not-executed instructions as compatibility annotations until their runtime lowering is implemented.

23 Quick-reference grammar

This is a practical description of the accepted surface language rather than a separate formal language specification:

```

protocol      := [params-line] logical-line*
params-line   := "PARAMS:" (name ":" type)*
logical-line  := command | comment | blank

command       := opcode argument*
opcode        := supported-command
               | primitive-gate
               | "B" primitive-gate
               | "PHASE=" angle
               | "BPHASE=" angle

gate-row      := opcode "-I" input+ "-O" output+ ["-$R"]
measure-row   := "MEASURE" ["-I" input]
set-row       := "SET" target value
process-row   := "MAIN-PROCESS" name
child-row     := ("RUNCHILD" | "CALL") name
               ["-I" argument*] ["-O" argument*]
returns-row   := "RETURNVALS" token*
accept-row    := "ACCEPTVALS" token*

token         := ["$"] name [":" suffix]
constant      := ("Op" | "Ip" | "Sp") ["_dim" integer]
angle         := rational
               | rational "d"
               | [sign] [integer ["*"]] "pi" ["/" integer]

```

Syntax detail: PHASE= pi/2 is joined into one opcode/value pair, but PHASE = pi/2 is not. Operations and flags are case-insensitive; names, types, and positional values are otherwise retained.

24 Glossary

Term	Meaning
Amplitude	A complex coefficient of a basis state. Its squared magnitude contributes to measurement probability.
Basis state	A state used as a coordinate direction. The computational basis for one qubit is $ 0\rangle, 1\rangle$.
Bloch sphere	A geometric representation of pure one-qubit states after ignoring global phase. North/south are $ 0\rangle/ 1\rangle$; longitude represents relative phase.
Child process	A separately named protocol expanded into a parent through RUNCHILD or CALL.
Control	A wire whose basis condition decides whether a controlled operation acts on its target. The control itself is preserved by the standard controlled gates.
Entanglement	A joint multi-qubit state that cannot be written as a tensor product of independent qubit states.
Gate	A state transformation. Closed-system quantum gates are unitary; measurement and reset have different semantics.
Global phase	A unit complex multiplier applied to the whole state. It does not change physical measurement predictions.
Ket	Dirac notation $ \psi\rangle$ for a state column vector.
Measurement	Conversion of quantum amplitudes into a sampled classical result, with state collapse and renormalization.

Term	Meaning
Phase	The angular part of a complex amplitude. Relative phase controls interference even when immediate basis probabilities are unchanged.
Process	A named QPU protocol body with ordered PARAMS and RETURNVALS.
Protocol	Plain-text QPU instructions describing preparation, gates, composition, measurement, and returned references.
Qubit	A normalized two-component complex state $\alpha 0\rangle + \beta 1\rangle$, not merely an unknown classical bit.
Register	One or more logical wires considered together. Current JOIN/SPLIT syntax is accepted but does not yet lower to register operations.
Relative phase	The phase difference between amplitudes. Unlike global phase, it can change later interference and measurement probabilities.
Superposition	A coherent linear combination of basis states. <code>sp</code> prepares the equal plus superposition.
Target	The wire whose amplitudes a gate mutates; normally the first <code>-0</code> reference.
Tensor product	The operation $\otimes$ that combines state spaces. Two one-qubit vectors produce a four-amplitude two-qubit vector.
Token	A source-level logical name resolved to a qubit wire or alias during compilation.
Truth table	Listed input/output expectations for measured RETURNVALS. It describes basis behavior but not arbitrary phase or entanglement.
Unitary	A matrix satisfying $U^\dagger U = I$, preserving normalization and having a reversible inverse.

25 Operation checklist

Item	Covered in	Item	Covered in
PARAMS:	Lexical syntax	MAIN-PROCESS	Process composition
X,Y,Z,H	Single-qubit gates	S,T,PHASE	Single-qubit gates
CNOT,CCNOT	Controlled gates	CZ,CY,SWAP	Controlled gates
NOT,AND,NAND	Derived gates	OR,XOR	Derived gates
MEASURE	Measurement	SET	State/token commands
CREATETOKEN	State/token commands	DELETETOKEN,FREE	State/token commands
INCREASECYCLE	State/token commands	JOIN,SPLIT	State/token commands
DECLARECHILD	Process composition	RUNCHILD,CALL	Process composition
RETURNVALS	Process composition	ACCEPTVALS	Process composition
COMPILEPROCESS	Compatibility	MASTERVAL	Compatibility
SAVE_STATE	Compatibility	LOAD_STATE	Compatibility
-I,-0,-\$R	Flags	B prefix	Single-qubit/reverse syntax
comments,\	Lexical syntax	.qpuio	Truth-table syntax